



UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO

Corso di Laurea Magistrale in Data Science
Gestione e Analisi di Big Data

A.A. 2025-2026

London Energy Pulse: Piattaforma di Streaming Analytics per Smart City

Professore Gianvito Pio
Studente Emanuele Piemontese

Emanuele Piemontese

837858

e.piemontese@studenti.uniba.it

INDICE

1. Introduzione e Obiettivi.....	4
1.1 Contesto	4
1.2 Il Problema	4
1.3 Obiettivo del progetto.....	4
2. Architettura del sistema	5
2.1 Architettura	6
2.1.1 Il Layer di Ingestione: Apache Kafka	6
2.1.2 Il Motore di Calcolo: Spark Structured Streaming & Scala	6
2.1.3 Il Layer di Persistenza: Medallion Architecture su Delta Lake	7
2.1.4 RealTimeAlertManager	7
2.1.5 BusinessInsightsEngine	8
2.1.6 Optimizer	8
3. Analisi dei Dataset	8
3.1 La sorgente: LCL SmartMeter Dataset.....	8
3.2 L'Arricchimento: gruppi Acorn	8
3.3 Il contesto ambientale: Weather Data.....	9
3.4 Analisi Esplorativa dei Dati (EDA).....	9
4. Ingestione dei Dati (Data Ingestion Layer).....	11
4.1 Architettura basata su Apache Kafka	11
4.2 Strategia di Ingestione: Historical Data Replay	11
4.3 Definizione del flusso	12
5. Elaborazione in Streaming (Processing Layer).....	13
5.1 Architettura della Pipeline: dal Bronze al Gold Layer	13
5.2 Pre-processing dei dati (Silver Layer)	13
5.3 Streaming Enrichment tramite Broadcast Join	15
5.4 Analisi Temporale: Watermarking e Windowing (Gold Layer)	15
5.5 Multi-target storage e Fault Tolerance	16
6. Intelligence & Stateful Analysis	17
6.1 Implementazione dello Stateful Streaming.....	17
6.2 Calcolo Statistico Incrementale: Algoritmo di Welford	17
6.3 Motore di Rilevamento delle Anomalie (Context-Aware)	19

6.4 Gestione del ciclo di vita e Fault Tolerance	20
6.5 Validazione e Testing	20
7. Monitoring Real-time, Business Intelligence e Maintenance	21
7.1 Gestione reattiva degli Alert	21
7.2 Business Insights Engine	22
7.3 Ottimizzazione e Manutenzione del Data Lake.....	23
8. Risultati e Conclusioni.....	23

1. Introduzione e Obiettivi

1.1 Contesto

Oggi, le città non sono più solo agglomerati di edifici, ma dei veri e propri ecosistemi digitali che generano flussi costanti di dati. In questo scenario, la gestione dell'energia rappresenta una delle sfide più critiche. Quello che si sta verificando è una transizione verso le **Smart Grid** e tutto questo richiede sistemi capaci di monitorare i consumi non più mensilmente, ma al secondo.

Il progetto "**London Energy Pulse**" nasce proprio con l'idea di smettere di guardare i dati energetici come delle fotografie statiche del passato (approccio *batch*) ma iniziare a trattare questi dati come un organismo vivo che pulsa in tempo reale (approccio *streaming*).

1.2 Il Problema

Fino a pochi anni fa, l'analisi dei consumi veniva effettuata tramite processi di "Data Mining" tradizionale su dataset storici. Sebbene utili per statistiche a lungo termine, come la pianificazione, questi modelli falliscono nel rispondere a domande cruciali per un operatore di rete moderno:

- *C'è un picco di carico anomalo proprio ora in un distretto specifico?*
- *Come sta reagendo il consumo di una fascia sociale all'improvviso calo della temperatura?*

La latenza dei sistemi tradizionali impedisce una risposta tempestiva, rendendo necessaria un'architettura che faccia del **tempo reale** il suo pilastro fondamentale.

1.3 Obiettivo del progetto

L'obiettivo principale di questo lavoro è la progettazione e l'implementazione di una pipeline di **Streaming Analytics**. Utilizzando il dataset **LCL (London DataStore)**, che raccoglie le letture di circa 5.500 famiglie londinesi, il sistema si pone i seguenti traguardi:

- **Ingestione ad alte prestazioni:** simulare un ambiente IoT urbano dove migliaia di sensori inviano dati simultaneamente tramite **Apache Kafka**;
- **Data Enrichment dinamico:** integrare istantaneamente i consumi con variabili esterne (es. meteo) e metadati socio-economici (Acorn Groups) utilizzando **Apache Spark**;
- **Intelligence reattiva:** sviluppare un sistema di **Stateful Anomaly Detection** capace di riconoscere derive comportamentali o guasti confrontando il dato istantaneo con il profilo storico dell'utente;
- **Resilienza e Consistenza:** garantire che il dato, una volta processato, sia archiviato in modo sicuro e interrogabile seguendo i principi della **Medallion Architecture** su **Delta Lake**.

2. Architettura del sistema

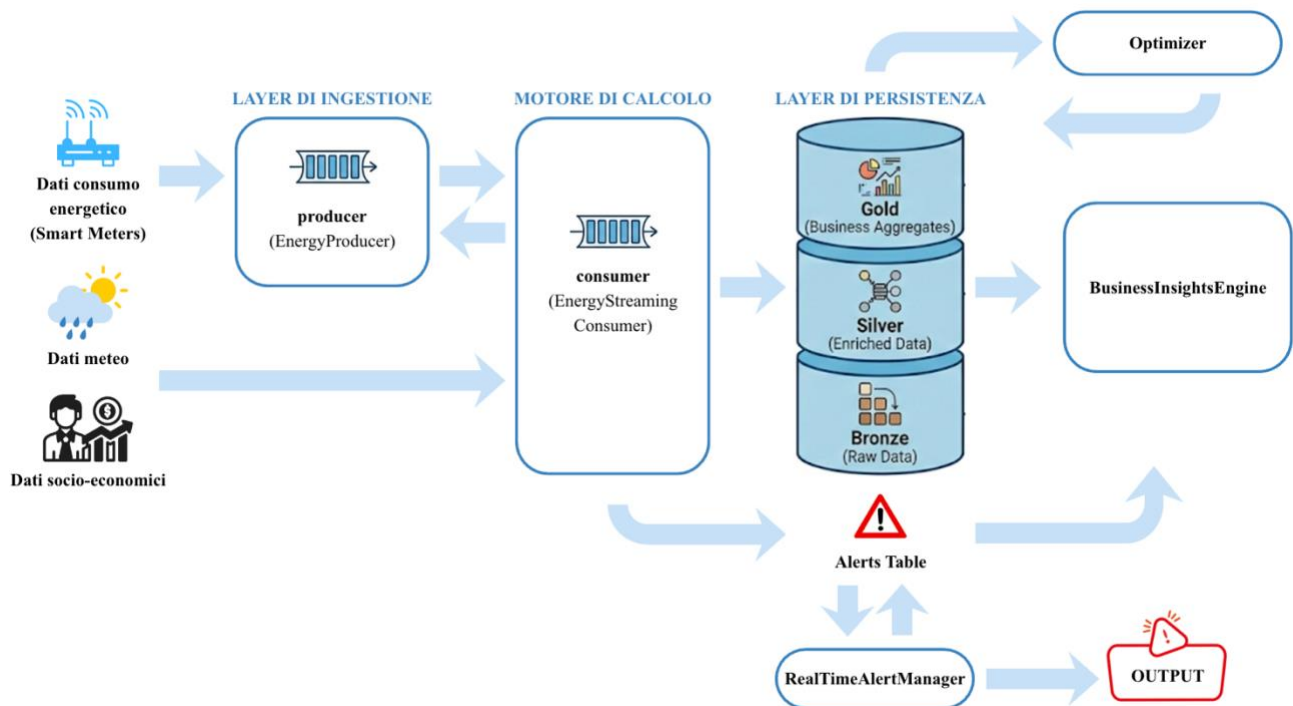
L'architettura di **London Energy Pulse** è progettata per essere scalabile, resiliente e "low-latency". Invece di una pipeline lineare, ho optato per un modello **Decoupled (Disaccoppiato)**. Questo significa che chi produce i dati (i sensori/Smart Meters) non sa chi li elaborerà (Spark). Questo disaccoppiamento è garantito da **Apache Kafka**. L'intero ecosistema segue i principi della **Medallion Architecture**, un pattern moderno che organizza i dati in diversi livelli di qualità e raffinamento, garantendo che l'informazione finale sia sempre affidabile e pronta per il business.

L'ecosistema del progetto è gestito tramite **SBT (Scala Build Tool) versione 1.9.8**, che si occupa della compilazione del codice e della risoluzione automatica delle dipendenze. Il linguaggio di riferimento è **Scala 2.12.18**, una versione scelta per la sua comprovata stabilità e compatibilità nativa con **Apache Spark 3.5.7**, che costituisce il motore di calcolo principale dell'intera pipeline.

All'interno del file di configurazione **build.sbt**, sono state definite le librerie di base dell'ecosistema Spark (**spark-core**, **spark-sql**, **spark-streaming** e **spark-mllib**) necessarie per l'elaborazione distribuita dei DataFrame e per l'analisi dei dati in tempo reale. Tuttavia, per abilitare la comunicazione con l'infrastruttura esterna e garantire la persistenza avanzata dei dati, lo stack è stato esteso con i seguenti moduli critici:

- **Integrazione Kafka (spark-sql-kafka-0-10)**: fornisce il connettore nativo che permette a Spark Structured Streaming di iscriversi ai topic Kafka e leggere/scrivere flussi di dati con latenze minime e garanzie di *exactly-once fault-tolerance*.
- **Delta Lake (delta-spark)**: componente indispensabile per implementare la *Medallion Architecture*. Questa libreria trasforma un normale storage in un Data Lakehouse, abilitando le transazioni ACID, l'enforcement degli schemi e la gestione sicura degli aggiornamenti concorrenti.
- **Serializzazione e Logging (jackson-module-scala e logback)**: poiché i messaggi provenienti dagli Smart Meter viaggiano in formato JSON, la libreria Jackson garantisce un parsing rapido e *type-safe* dei payload. Infine, l'inclusione di un framework di logging permette di isolare i log applicativi da quelli di sistema di Spark, facilitando il monitoraggio.

2.1 Architettura



2.1.1 Il Layer di Ingestione: Apache Kafka

Il punto di ingresso è gestito dal componente `EnergyProducer`, il quale riceve i dati granulari dai **Smart Meters** in tempo reale. Per gestire migliaia di messaggi al secondo senza perdere informazioni, ho scelto Kafka. In questo progetto, Kafka non è un semplice "postino", ma un buffer intelligente che gestisce la **Backpressure**: se Spark dovesse rallentare, Kafka trattiene i dati in sicurezza finché il sistema non è pronto a processarli.

Il motore di calcolo fonde il flusso dinamico con dataset statici/semi-statici (Dati Meteo e Socio-economici ACORN) per contestualizzare ogni singola lettura energetica.

2.1.2 Il Motore di Calcolo: Spark Structured Streaming & Scala

Il cuore pulsante del sistema è **Apache Spark**. La scelta è ricaduta sulle **Structured Streaming API** per la loro capacità di trattare i flussi di dati come se fossero tabelle infinite, semplificando enormemente la scrittura del codice.

Invece, **Scala** è stato scelto per:

1. **Performance:** essendo Spark scritto in Scala, l'integrazione è nativa e le performance sono superiori rispetto a Python (PySpark), specialmente nelle operazioni complesse.

2. **Type Safety:** in un contesto Smart City, un errore di tipo (es. una stringa dove ci si aspetta un decimale) può far crollare l'intera pipeline. La tipizzazione forte di Scala previene questi bug a tempo di compilazione.

Spark esegue le seguenti operazioni critiche:

- **Pre-processing:** effettua operazioni di pulizia, gestione dei dati.
- **Stream-Batch Join (enrichment):** unisce il flusso dei consumi con quello del meteo.
- **Watermarking:** gestisce i dati che arrivano "fuori tempo" o in ritardo a causa di problemi di rete simulati.
- **Stateful analysis:** grazie a mapGroupsWithState, Spark mantiene in memoria uno "stato" per ogni casa, permettendoci di fare analisi storiche istantanee senza interrogare un database esterno.

2.1.3 Il Layer di Persistenza: Medallion Architecture su Delta Lake

Un problema comune nei sistemi di streaming è la "corruzione" dei file se un processo si interrompe. Per risolvere questo, ho implementato **Delta Lake**, che porta le transazioni **ACID** nel mondo dei Big Data. Ho organizzato i dati in tre livelli logici:

1. **Livello Bronze (Raw):** i dati grezzi provenienti da Kafka. È la "scatola nera": se qualcosa va storto nell'analisi, è possibile sempre ripartire da qui.
2. **Livello Silver (Enriched):** qui i dati sono puliti, filtrati dai null e uniti alle informazioni meteorologiche e socio-economiche (Acorn Groups). È il set di dati perfetto per i data scientist.
3. **Livello Gold (Business):** contiene aggregazioni pre-calcolate (es. consumo medio orario per quartiere). Queste tabelle sono ottimizzate per essere lette da dashboard (come Grafana o Tableau) o query SQL veloci.

2.1.4 RealTimeAlertManager

È il modulo di **reattività operativa del sistema**. Sfruttando l'elaborazione incrementale di Spark Streaming, il manager **intercetta in tempo reale i nuovi record di anomalia** generati dal Consumer, senza appesantire la pipeline principale di ingestione dati. Questo approccio garantisce che ogni evento(alert) venga immediatamente trasformato in un output concreto, sia esso un avviso granulare per i tecnici o l'attivazione di procedure automatiche di sicurezza per la salvaguardia dell'infrastruttura elettrica.

2.1.5 BusinessInsightsEngine

Motore analitico che interroga i layer Silver e Gold per estrarre trend socio-economici e tutte le tipologie di informazioni nascoste dai dati per prendere decisioni di business.

2.1.6 Optimizer

Un modulo di manutenzione automatizzata che esegue operazioni di **optimize** e **vacuum**. Questo garantisce che le prestazioni delle query rimangano elevate nonostante il flusso continuo di dati in streaming.

3. Analisi dei Dataset

3.1 La sorgente: LCL SmartMeter Dataset

Il cuore pulsante del progetto è il dataset **Low Carbon London (LCL)**, ospitato sul London DataStore e ora disponibile su Kaggle. Non si tratta di un semplice file di prova, ma di un archivio massivo che contiene le letture dei contatori intelligenti di oltre **5.500 famiglie londinesi** tra il 2011 e il 2014.

Il dataset è costituito da oltre **167 milioni di righe**. Ogni record rappresenta il consumo di energia elettrica rilevato ogni 30 minuti (half-hourly).

- **ID Utente (LCLid):** l'identificativo univoco dell'abitazione.
- **Timestamp:** data e ora della lettura.
- **KWh:** il consumo effettivo registrato nel periodo.

Il dataset è suddiviso in 112 file CSV denominati:

- block_0.csv
- block_1.csv
- block_2.csv
- ...

Ogni file *block* contiene una porzione del dataset complessivo (circa 50 famiglie per blocco).

3.2 L'Arricchimento: gruppi Acorn

I dati di consumo, da soli, dicono solo *quanto* viene consumato. Per capire il *perché*, ho integrato i metadati **Acorn**. Si tratta di una classificazione socio-demografica (sviluppata da CACI) che suddivide la popolazione in categorie basate sullo stile di vita e sul benessere economico.

Nel progetto, queste informazioni ci permettono di segmentare l'analisi:

- **Affluent:** utenti con alta disponibilità economica;
- **Comfortable:** la classe media;
- **Adversity:** fasce di popolazione più vulnerabili.

Questa dimensione è fondamentale per l'Anomaly Detection: un picco di consumo in una zona "Affluent" potrebbe avere cause molto diverse rispetto a uno in una zona "Adversity".

3.3 Il contesto ambientale: Weather Data

Il terzo pilastro dei dati è costituito dalle **informazioni meteorologiche storiche di Londra** (temperatura, umidità, velocità del vento). È risaputo che il consumo energetico è fortemente correlato alla temperatura esterna:

- in inverno, il calo della temperatura spinge l'uso del riscaldamento elettrico.
- in estate, ondate di calore possono causare picchi dovuti ai sistemi di climatizzazione.

Integrando questi dati nello streaming, Spark può "giustificare" un aumento improvviso dei consumi, evitando di segnalare falsi positivi come anomalie quando in realtà si tratta solo di una giornata particolarmente fredda.

3.4 Analisi Esplorativa dei Dati (EDA)

Ho effettuato un'analisi esplorativa esaustiva sull'intero corpo del dataset, composto da **167.817.021 di record**. L'elaborazione, eseguita in modalità distribuita tramite Apache Spark, ha permesso di mappare l'intero perimetro temporale dei consumi, che si estende dal **23 novembre 2011 al 28 febbraio 2014**.

L'analisi ha evidenziato un'elevata coerenza strutturale: il formato dei timestamp è risultato perfettamente consistente su scala globale, con una lunghezza costante di **27 caratteri**, eliminando il rischio di perdite di dati durante le operazioni di casting temporale. Per quanto riguarda la pulizia dei dati, sono stati individuati **5.560 record** con valori di consumo mancanti o etichettati come stringhe "Null", una quota estremamente ridotta rispetto al volume totale (circa lo 0,003%), che permette una gestione tramite filtraggio semplice senza compromettere la significatività statistica.

Un risultato di particolare rilievo riguarda la **verifica dell'integrità referenziale**: il confronto tra il dataset energetico e i metadati delle utenze ha restituito **zero record orfani**, confermando che ogni misurazione nel dataset è riconducibile a un'abitazione censita. Infine, l'analisi dei gruppi socio-economici ha confermato la presenza di una stratificazione basata su categorie ACORN, individuando tuttavia una trascurabile percentuale di etichette spurie (ACORN-U, ACORN-) che verranno

normalizzate nella successiva fase di preprocessing. Questo screening completo assicura che lo schema definito per la pipeline di streaming sia resiliente e privo di ambiguità strutturali.

Risultati EDA

=== Qualità dei Dati (Null o stringhe 'Null') ===

Missing_Energy	Missing_Timestamp	Missing_LCLid
5.560	0	0

=== Top 10 Valori Consumo Grezzi ===

Energy (kWh/hh)	Frequenza
-----------------	-----------

0.000	2.001.552
0.053	912.931
0.054	912.008
0.052	911.794
0.048	911.581
0.047	910.050
0.055	909.754
0.051	907.222
0.056	906.548
0.046	905.736

only showing top 10 rows

=== Estensione Temporale del Dataset (Min/Max) ===

Data Inizio	Data Fine
-------------	-----------

23/11/2011 09:00	28/02/2014 00:00
------------------	------------------

=== Consistenza Formato Timestamp (Lunghezza stringa) ===

Lunghezza Stringa	Timestamp	Frequenza
-------------------	-----------	-----------

27 caratteri		167.817.021
--------------	--	-------------

=== Distribuzione Gruppi Acorn (Intera Popolazione) ===

Acorn_Grouped	Numero Abitazioni
---------------	-------------------

Affluent	2.192
Adversity	1.816
Comfortable	1.507
ACORN-U	49
ACORN-	2

4. Ingestione dei Dati (Data Ingestion Layer)

La fase di **Data Ingestion** rappresenta il punto di transizione del progetto da un'analisi statica di tipo *batch* a un paradigma di elaborazione **Near Real-Time**. In un ecosistema di Smart Grid, i dati non risiedono in archivi statici, ma fluiscono continuamente dai sensori (smart meters) verso i centri di elaborazione.

4.1 Architettura basata su Apache Kafka

Per gestire il flusso di dati ho scelto **Apache Kafka** come *message broker distribuito*. Kafka agisce come il "sistema nervoso" dell'architettura, svolgendo il ruolo di buffer transazionale tra la sorgente dei dati e il motore di calcolo.

Struttura:

- **Producer:** uno script Scala che legge i CSV e "spara" i dati dentro Kafka.
- **Topic:** una sorta di "canale" (*energy-readings*) dove i dati circolano in attesa di essere letti.
- **Consumer (Spark):** l'applicazione Spark Structured Streaming che "ascolta" il canale e processa i dati man mano che arrivano.

L'integrazione di Kafka offre tre vantaggi strutturali decisivi:

1. **Disaccoppiamento (Decoupling):** il sistema che genera i dati (Producer) e quello che li analizza (Consumer) sono totalmente indipendenti; un eventuale rallentamento della pipeline di analisi non influisce sulla ricezione dei dati dai contatori.
2. **Persistenza e Fault Tolerance:** Kafka memorizza i messaggi su disco per un periodo configurabile (retention), permettendo il recupero dei dati in caso di crash dei consumatori senza perdita di informazioni.
3. **Scalabilità Orizzontale:** grazie al partizionamento dei *Topic*, il sistema può gestire incrementi massivi del volume di traffico semplicemente aggiungendo nuovi nodi al cluster.

4.2 Strategia di Ingestione: Historical Data Replay

Dato che il dataset originale è composto da file storici in formato CSV, è stato sviluppato un **Kafka Producer** personalizzato in Scala. A differenza di un semplice caricamento massivo, il Producer implementa una logica di **Historical Replay**.

Il Producer elabora i file CSV in ordine sequenziale (es. *block_1.csv* con ID 1-50, seguito da *block_2.csv* con ID 51-100). Per garantire che il sistema di monitoraggio riceva i dati in modo coerente, sono stati implementati due meccanismi chiave:

- **Ordinamento sequenziale:** ogni file viene letto riga per riga, preservando l'ordine cronologico originale delle letture per ogni singolo contatore.
- **Partizionamento per chiave (LCLid):** utilizzando l'identificativo del contatore come chiave del messaggio Kafka, il sistema garantisce che tutte le letture di un determinato utente siano inviate alla stessa partizione. Questo permette al **Consumer** di processare la storia di ogni sensore in ordine temporale perfetto, condizione indispensabile per la corretta evoluzione dell'**Algoritmo di Welford**.

Nota Tecnica sull'Historical Replay: sebbene nel mondo reale gli smart meter inviino dati ogni 30 minuti, il Producer implementa un **Replay Accelerato**. L'ordine cronologico dei timestamp è preservato a livello di singola utenza per non inficiare la validità statistica dello Z-Score, ma la frequenza di invio è stata aumentata per testare la resilienza e il *throughput* dell'intera architettura Big Data.

Questo approccio permette di validare in pochi minuti analisi che, in tempo reale, richiederebbero mesi di osservazione, simulando l'inserimento progressivo di nuovi "cluster" di utenze (i blocchi) nella rete monitorata.

4.3 Definizione del flusso

Il flusso di ingestione è il seguente:

1. **Source (Raw Bronze):** file CSV distribuiti contenenti le letture storiche grezze. Questi file rappresentano il punto di partenza dell'intera pipeline e non hanno subito alcuna trasformazione preventiva, simulando la natura eterogenea dei dati IoT.
2. **Ingestion Engine (Producer):** uno script sviluppato in Scala utilizzando la libreria kafka-clients. Il suo compito è il parsing riga per riga dei CSV, la serializzazione dei record in formato **JSON** e l'invio al broker. Implementa la logica di *Historical Replay* per simulare la frequenza temporale dei contatori.
3. **Message Broker (Kafka):** un cluster Apache Kafka locale configurato con il topic energy-readings. Agisce come buffer transazionale ad alta disponibilità, garantendo che i dati siano persistenti e pronti per essere consumati in modalità "push" o "pull".
4. **Stream Processor & Sink (Silver Layer):** un'applicazione **Spark Structured Streaming** che agisce come consumatore del topic. In questa fase avviene la trasformazione "on-the-fly": i dati JSON vengono validati, puliti e arricchiti con i metadati (Acorn e Weather) prima di essere salvati definitivamente nel **Silver/Gold Layer** in formato Delta Lake.

5. Elaborazione in Streaming (Processing Layer)

Il cuore tecnologico del sistema risiede nel **Processing Layer**, dove i dati grezzi provenienti da Kafka vengono trasformati in informazioni strutturate e arricchite. Per questa fase ho utilizzato **Spark Structured Streaming**, un motore di elaborazione distribuita che permette di trattare i flussi di dati in tempo reale con la stessa semantica di un'elaborazione batch.

5.1 Architettura della Pipeline: dal Bronze al Gold Layer

L'elaborazione segue un'evoluzione del paradigma **Medallion Architecture**, gestendo simultaneamente la persistenza dei dati atomici e la generazione di metriche aggregate.

- **Ingestione (Bronze Layer):** i messaggi vengono consumati dal topic Kafka `energy-readings`. In questa fase, il dato è in formato binario/JSON e richiede validazione.
- **Arricchimento (Silver Layer):** il dato viene normalizzato e unito a dataset statici (Meteo e Households) tramite *Broadcast Join*.
- **Analisi e KPI (Gold Layer):** viene applicata la logica di *Windowing* e *Watermarking* per produrre medie mobili e carichi di rete.

Il consumer esegue una serie di operazioni in streaming, tra cui:

5.2 Pre-processing dei dati (Silver Layer)

Prima di procedere all'analisi, il flusso streaming subisce una serie di trasformazioni atte a garantire l'integrità del dato:

B. Dataset Metadati Acorn

- **Integrità referenziale:** è stata applicata una `dropDuplicates` preventiva sull'ID utente come misura di sicurezza per garantire un rapporto 1:1 durante il join.
- **Filtraggio categoriale:** sono state rimosse le categorie non informative (ACORN-U, ACORN-) per mantenere la purezza statistica dei gruppi socio-economici.

=== Esempio di record dopo pre-processing metadati ===

Meter_ID Acorn_Group

MAC000002 Affluent

MAC000003 Adversity

MAC000004 Affluent

MAC000005 Affluent

MAC000006 Adversity

C. Dataset Weather (Dati Contestuali)

- **Feature selection:** sono state selezionate solo le variabili con impatto provato sul consumo energetico: `temperature`, `apparentTemperature` (*percepita*), `humidity` e `precipType`.

=== Esempio di record dopo pre-processing meteo ===

Weather_Hour	Temperature	Apparent Temperature	Humidity	PrecipType
2011-11-11 00:00:00	10.24	10.24	0.91	rain
2011-11-11 01:00:00	9.76	8.24	0.94	rain
2011-11-11 02:00:00	9.46	7.76	0.96	rain
2011-11-11 03:00:00	9.23	7.44	0.96	rain
2011-11-11 04:00:00	9.26	7.24	1.00	rain

A. Dataset Consumi (data stream)

- **Parsing JSON:** poiché Kafka trasmette i dati come payload binari, è stato applicato uno schema predefinito (`energySchema`) per estrarre i campi `LCLid`, `tstp` (timestamp) e `energy`.
- **Normalizzazione temporale:** la colonna `tstp` è stata convertita in Timestamp utilizzando un parsing specifico a **27 caratteri** (scoperto in fase di EDA). Questo garantisce la precisione al microsecondo necessaria per il futuro *Watermarking* in streaming.
- **Schema enforcement:** il consumo è stato castato a Double. Le righe contenenti stringhe "Null" o campi vuoti (circa lo 0,003% del totale) vengono filtrate per prevenire errori nei calcoli matematici.
- **Resampling preparation:** è stata generata una colonna `join_hour` tramite la funzione `date_trunc`, fondamentale per allineare i consumi semi-orari (30 minuti) con i dati meteo orari.

=== Schema del flusso in ingresso da Kafka ===

root

```
-- key: binary (nullable = true)
-- value: binary (nullable = true)
-- topic: string (nullable = true)
-- partition: integer (nullable = true)
-- offset: long (nullable = true)
-- timestamp: timestamp (nullable = true)
-- timestampType: integer (nullable = true)
```

5.3 Streaming Enrichment tramite Broadcast Join

Una delle sfide principali di questo progetto è l'arricchimento del dato in tempo reale. Per ogni lettura energetica ricevuta, il sistema deve conoscere il contesto socio-economico dell'utente e le condizioni meteorologiche correnti.

Per ottimizzare le prestazioni, è stata utilizzata la tecnica del **Broadcast Join**. I dataset statici (Metadati Households e Meteo) vengono distribuiti nella memoria di ogni nodo *executor* di Spark. Questo evita il costoso spostamento di dati in rete (*shuffling*) durante il join con il flusso principale.

- **Join con profilo sociale (Acorn):** ogni record è stato unito al dataset delle abitazioni tramite la chiave LCLid, aggiungendo la categorizzazione ACORN (es. *Affluent*, *Comfortable*, *Adversity*).
- **Join meteorologico (Allineamento Temporale):** poiché i dati energetici hanno frequenza semi-oraria e quelli meteo oraria, è stata creata una colonna di appoggio (`join_hour`) tramite la funzione `date_trunc`. Questo ha permesso di associare a ogni consumo la temperatura e l'umidità specifica di quell'ora.

Esempio di record arricchito e normalizzato:

LCLid	Timestamp	Consumption	Acorn_Group	Temperature	Apparent Temperature	Humidity	Precip Type
MAC000048	2011-12-08 12:30:00	0.229	Affluent	11.47	11.47	0.80	rain
MAC000048	2011-12-08 13:00:00	0.213	Affluent	11.82	11.82	0.82	rain
MAC000048	2011-12-08 13:30:00	0.272	Affluent	11.82	11.82	0.82	rain
MAC000048	2011-12-08 14:00:00	0.576	Affluent	11.52	11.52	0.86	rain
MAC000048	2011-12-08 14:30:00	0.194	Affluent	11.52	11.52	0.86	rain

5.4 Analisi Temporale: Watermarking e Windowing (Gold Layer)

Per trasformare i dati atomici in informazioni strategiche, sono state implementate logiche di aggregazione temporale avanzate.

- **Watermarking:** ho definito una soglia di tolleranza di **10 minuti** per la gestione dei dati in ritardo (*Late Data*). Spark mantiene lo stato della finestra temporale finché il watermark non supera il limite impostato, garantendo l'accuratezza delle medie anche in caso di latenze di rete dei sensori.

- **Sliding Windows:** sono state configurate finestre temporali di **1 ora**, scorrevoli ogni **30 minuti**. Questo permette di calcolare una "media mobile" del consumo e del carico totale, identificando i trend di crescita o decrescita della domanda energetica in tempo reale.

5.5 Multi-target storage e Fault Tolerance

Il sistema implementa una scrittura simultanea su due destinazioni (*Multi-Sink*) per massimizzare l'utilità dei dati:

1. **Silver Layer (Delta Lake - Append Mode):** persistenza di ogni singolo record arricchito. Utile per analisi forensi, audit e machine learning granulare.
2. **Gold Layer (Delta Lake - Complete Mode):** salvataggio dei KPI aggregati (medie di consumo, carico totale, case attive).

Il risultato non viene salvato in semplici file CSV o Parquet, ma in una tabella **Delta Lake**. L'uso di Delta Lake offre vantaggi cruciali per una Smart Grid:

- **Transazioni ACID:** garantisce che le letture parziali dei dati non avvengano mai durante una fase di scrittura.
- **Schema Enforcement:** impedisce che dati con formati errati "sporchino" il database Silver.
- **Time Travel:** permette di interrogare versioni precedenti del dataset per scopi di auditing o analisi storica.

Per garantire che il sistema sia resiliente ai guasti, è stato implementato il meccanismo di **Checkpointing**.

Spark salva costantemente lo stato dell'elaborazione (offset di Kafka e metadati di scrittura) in una directory dedicata. In caso di arresto anomalo, al riavvio il sistema è in grado di recuperare l'esatto punto di interruzione, garantendo la semantica **Exactly-Once**: ogni dato viene elaborato e scritto nel database finale una e una sola volta, evitando duplicazioni o perdite di informazioni.

6. Intelligence & Stateful Analysis

Il cuore del sistema è l'implementazione di un'analisi di tipo *stateful*, capace di apprendere il comportamento energetico di ogni singola utenza in tempo reale.

6.1 Implementazione dello Stateful Streaming

Per superare i limiti delle aggregazioni temporali standard (che "dimenticano" il dato una volta chiusa la finestra), ho utilizzato l'operazione `flatMapGroupsWithState`. Questa astrazione di Spark Structured Streaming permette di mantenere in memoria un oggetto di stato (`MeterState`) per ogni chiave (`LCLid`), rendendo possibile il tracciamento dell'evoluzione storica del consumo senza la necessità di interrogare database esterni ad ogni micro-batch.

Il sistema si basa su tre entità fondamentali definite tramite *Case Classes* in Scala:

1. **MeterReading**: modello di input che incapsula la lettura energetica arricchita con il dato meteorologico locale;
2. **MeterState**: il contenitore dello stato persistente. Memorizza la media mobile (μ), la somma dei quadrati delle differenze ($M2$), il conteggio delle letture (n) e il riferimento temporale dell'ultima attività;
3. **AnomalyAlert**: il modello di output generato solo al verificarsi di violazioni delle soglie statistiche e fisiche preimpostate.

6.2 Calcolo Statistico Incrementale: Algoritmo di Welford

Il calcolo della varianza su flussi di dati potenzialmente infiniti presenta sfide di stabilità numerica e gestione della memoria. È stato quindi implementato l'**Algoritmo di Welford**, che permette l'aggiornamento dei parametri statistici in un unico passaggio (*one-pass*) con complessità computazionale $O(1)$.

Tradizionalmente, per calcolare la varianza σ^2 di un set di dati, bisogna prima conoscere la media di **tutti** i numeri, poi sottrarre tale media da ogni singolo numero, elevarlo al quadrato e sommare tutto. Questo richiede due passaggi sui dati (due "passate").

L'**algoritmo di Welford**, invece, aggiorna i valori **incrementalmente** ogni volta che arriva un nuovo dato x_n . Utilizza tre variabili di stato:

1. **n** : il numero di campioni visti finora.
2. **μ_n** : la media corrente.
3. **$M2_n$** : la somma dei quadrati delle differenze dalla media corrente (utilizzata per derivare la varianza).

Ogni volta che arriva un nuovo valore x , l'algoritmo esegue questi passaggi:

1. **Incremento del conteggio**: $n = n + 1$
2. **Calcolo del Delta**: $\Delta = x - \mu_{n-1}$ (differenza tra nuovo dato e media precedente)
3. **Aggiornamento Media**: $\mu_n = \mu_{n-1} + \frac{\Delta}{n}$
4. **Aggiornamento Somma Quadrati ($M2$)**: $M2_n = M2_{n-1} + \Delta \times (x - \mu_n)$
5. **Varianza Finale**: $\sigma^2 = \frac{M2_n}{n-1}$

La scelta di questo algoritmo è dettata da tre ragioni tecniche fondamentali:

A. Efficienza della memoria

In un sistema di streaming con migliaia di contatori (LCLid), non è possibile memorizzare l'intera cronologia dei consumi di ogni casa per ricalcolare la media ogni 30 minuti (limite dalla RAM).

Con Welford: per ogni utente memorizzi solo **3 numeri** (n , media, $M2$). Non importa se si ricevono 10 o 10 milioni di letture, l'occupazione di memoria rimane costante.

B. Stabilità numerica

Il metodo classico "somma dei quadrati" ($\sum x^2 - (\sum x)^2 / n$) è vulnerabile alla cosiddetta **catastrophic cancellation**. Quando si lavora con numeri molto grandi (come i consumi totali di una città), la precisione dei decimali nei computer può generare errori enormi o varianze negative.

Con Welford: è intrinsecamente stabile perché lavora sulle differenze dei valori (Δ), mantenendo la precisione del calcolo anche dopo mesi di streaming ininterrotto.

C. Analisi Real-time

Spark Structured Streaming elabora dati in micro-batch. Welford si sposa perfettamente con l'operatore `mapGroupsWithState`. Ad ogni batch, Spark "risveglia" lo stato del contatore, applica la formula di Welford sul nuovo record e "richiude" lo stato. Questo permette un'Anomaly Detection istantanea, non bisogna aspettare la fine della giornata per sapere se un consumo è anomalo.

La deviazione standard (σ) viene derivata istantaneamente come:

$$\sigma = \sqrt{\frac{M2_n}{n-1}}$$

Questo approccio garantisce che il sistema non debba memorizzare l'intero storico delle letture, riducendo drasticamente l'occupazione della RAM sul cluster Spark.

6.3 Motore di Rilevamento delle Anomalie (Context-Aware)

La logica di rilevamento non è statica, ma si adatta dinamicamente al contesto ambientale e alla storia dell'utente attraverso un processo a tre fasi.

Per evitare falsi positivi causati da una base statistica insufficiente, è stata implementata una **clausola di warm-up**. Il sistema ignora le anomalie per le prime **600 letture** (corrispondenti a circa 12 giorni di dati per contatore). In questo intervallo, il sistema popola esclusivamente lo stato per costruire una baseline di consumo affidabile.

Il principale indicatore di anomalia è lo **Z-Score**, che rappresenta la distanza del consumo attuale dalla media storica espressa in unità di deviazione standard:

$$Z = \frac{|x - \mu|}{\sigma + 0.1}$$

Nota: la costante \$0.1\$ al denominatore previene la divisione per zero e stabilizza il calcolo per utenze con consumi estremamente costanti.

La soglia critica di Z viene modulata in base alla temperatura:

- **Contesto Standard ($6^{\circ}\text{C} \leq T \leq 30^{\circ}\text{C}$):** soglia impostata a $Z > 8$. Un picco improvviso in clima mite è considerato altamente sospetto. Scegliere un valore così alto (rispetto al classico $Z = 3$) serve a isolare solo eventi eccezionali, come guasti agli elettrodomestici o comportamenti umani rari, garantendo che il sistema non generi allarmi per piccoli cambiamenti di routine.
- **Contesto Estremo ($T < 6^{\circ}\text{C}$ o $T > 30^{\circ}\text{C}$):** la soglia viene elevata a $Z > 15$. Il sistema aumenta la propria tolleranza "comprendendo" che il picco è probabilmente dovuto a carichi termici (caldaie o condizionatori).

Oltre alla deviazione statistica, è stato imposto un vincolo fisico di magnitudo. Un alert viene generato solo se il consumo x supera la soglia di **6.0 kWh/hh**. Una lettura di 6.0 kWh in mezz'ora equivale a un assorbimento costante di **12 kW**. In un contesto residenziale standard (dove la potenza impegnata

è spesso inferiore), un tale valore rappresenta una criticità fisica che può indicare un cortocircuito, un malfunzionamento grave o un sovraccarico pericoloso per l'infrastruttura di distribuzione locale. Questo filtro garantisce che le segnalazioni siano limitate a eventi di reale criticità per la stabilità della rete elettrica o possibili guasti gravi, escludendo fluttuazioni minori.

6.4 Gestione del ciclo di vita e Fault Tolerance

La gestione dello stato è configurata per essere resiliente ed efficiente:

1. **Memory Eviction (Timeout):** è stato implementato un `ProcessingTimeTimeout` di **2 ore**. Se un contatore smette di inviare dati per tale periodo, lo stato viene rimosso dalla memoria. Questo meccanismo di *garbage collection* è essenziale per gestire il turnover delle utenze in una Smart Grid reale.
2. **Checkpointing:** lo stato viene sincronizzato periodicamente su Delta Lake. In caso di crash del sistema, Spark ripristina i valori di μ , $M2$ e n dall'ultimo checkpoint, garantendo la continuità dell'analisi senza perdere il "pregresso" di ogni utente.
3. **Ordinamento temporale:** prima dell'elaborazione, i dati all'interno di ogni micro-batch vengono ordinati cronologicamente (`sortBy(_.timestamp.getTime)`). Questo assicura che l'aggiornamento delle medie mobili segua la reale sequenza degli eventi, preservando la coerenza statistica.

6.5 Validazione e Testing

Per il progetto la fase di testing è stata focalizzata sulla logica di **Stateful Processing**, con i seguenti obiettivi:

1. **Validazione della stabilità numerica:** verificare che l'algoritmo di **Welford** aggiorni correttamente la media mobile e la somma dei quadrati ($M2$) senza derive computazionali nel tempo.
2. **Verifica delle soglie dinamiche:** accertare che il sistema applichi correttamente la logica *Context-Aware*, variando la sensibilità dello Z-Score (8 vs 15) in base alle condizioni meteorologiche simulate.
3. **Prevenzione dei falsi positivi:** testare che la fase di "calibrazione" (le prime 600 letture) inibisca correttamente la generazione di alert prematuri basati su una baseline statistica non ancora solida.

Data la natura complessa dell'interfaccia `GroupState` di Apache Spark, il test è stato isolato utilizzando **Mockito** per simulare il comportamento del motore di streaming. È stato simulato uno stato storico

(MeterState) per un contatore specifico, evitando di dover istanziare un intero cluster Spark durante i test unitari.

Scenario di anomalia: il test principale ha simulato l'arrivo di un consumo anomalo (es. 12.0 kWh) a fronte di una media consolidata di 1.0 kWh. Il successo del test ha confermato che:

- l'alert viene generato correttamente in condizioni climatiche miti;
- lo stato del contatore viene aggiornato incrementando il contatore delle letture e ricalcolando i parametri statistici per la finestra successiva.

7. Monitoring Real-time, Business Intelligence e Maintenance

7.1 Gestione reattiva degli Alert

Il modulo `RealTimeAlertManager` rappresenta il punto di contatto tra il sistema di analisi e l'operatore umano. È progettato per operare con una logica **event-driven**, reagendo istantaneamente alla scrittura di un nuovo record nella tabella degli alert.

A differenza dei database tradizionali, Delta Lake permette di utilizzare una tabella come sorgente per un flusso continuo. Il sistema esegue un polling ogni **5 secondi** sulla tabella `alerts_history`.

Vantaggio: questo approccio garantisce il disaccoppiamento (*decoupling*) tra il motore di analisi (Cap. 6) e il sistema di notifica, permettendo a entrambi di scalare indipendentemente.

La scelta tecnica di utilizzare l'operatore `.foreachBatch` è fondamentale per la flessibilità del sistema. All'interno di questa funzione, il sistema:

1. **Filtra i batch vuoti:** evita l'overhead computazionale e di log quando non ci sono nuove anomalie;
2. **Calcolo della latenza:** utilizzando `current_timestamp().as("processed_at")`, il sistema traccia il momento esatto della notifica, permettendo analisi sulla tempestività dell'intervento;
3. **Simulazione di Sink esterni:** in un'architettura di produzione, questo blocco di codice è il punto di integrazione naturale per API di terze parti (es. invio tramite **email**, **notifiche push** o **sistemi di alerting esterni**).

7.2 Business Insights Engine

L'analisi dei dati energetici è stata segmentata in tre layer diagnostici per fornire una visione a 360° della Smart Grid.

L'analisi sul **Silver Layer** mira a comprendere la psicologia del consumo e la risposta degli utenti agli stimoli esterni:

- **Sensibilità termica (Pearson Correlation):** viene calcolato il coefficiente di correlazione r tra temperatura e consumo.

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

Un valore vicino a 1 o -1 indica un'utenza il cui consumo è rigidamente dettato dal clima (es. riscaldamento elettrico).

- **Indice di volatilità (Coefficient of Variation - CoV):** definito come $CoV = \sigma / \mu$, identifica l'imprevedibilità di un gruppo Acorn. Un CoV elevato indica abitudini irregolari, rendendo più difficile la pianificazione del carico per il fornitore.
- **Weekend Shift Analysis:** attraverso operazioni di pivot, viene quantificata la variazione percentuale del carico tra giorni feriali e festivi, informazione cruciale per il dimensionamento delle riserve energetiche nel fine settimana.

Il **Gold Layer** fornisce metriche aggregate per la gestione dell'infrastruttura urbana:

- **Peak-to-Average Ratio (PAR):** è l'indicatore principale dello stress di rete.

$$PAR = \frac{\max(\text{Carico Totale})}{\text{avg}(\text{Carico Totale})}$$

Un PAR elevato suggerisce la necessità di incentivare lo spostamento del carico (*load shedding*) per evitare il sovraccarico dei trasformatori locali.

- **Efficienza per nodo:** calcola il consumo per "contatore attivo", permettendo di identificare quartieri con carichi anomali nonostante una bassa densità abitativa.

L'analisi degli alert non serve solo alla reazione, ma alla prevenzione futura:

- **Severity scoring:** rapporto tra l'entità del picco e la media storica. Aiuta a dare priorità agli interventi tecnici.
- **Root Cause Weather Analysis:** utilizzando la tecnica del "binning" (raggruppamento per fasce di temperatura), il sistema verifica se le anomalie tendono a concentrarsi in specifici range climatici, validando l'efficacia delle soglie dinamiche impostate nel Capitolo 6.

7.3 Ottimizzazione e Manutenzione del Data Lake

L'elaborazione in streaming produce una grande quantità di file di piccole dimensioni che possono rallentare le prestazioni di lettura nel tempo. Il modulo `Optimizer` gestisce la sostenibilità a lungo termine.

Il comando `deltaTable.optimize().executeCompaction()` istruisce Spark a riorganizzare i dati sul disco. I piccoli file JSON o Parquet generati dai micro-batch (ogni 10 secondi) vengono fusi in file più grandi e ottimizzati. Questo processo trasforma le letture da "migliaia di accessi casuali" a "poche letture sequenziali", migliorando le performance delle query di Business Intelligence fino a **10 volte**.

Delta Lake supporta il *Time Travel*, ovvero la possibilità di accedere a versioni passate dei dati. Tuttavia, questo richiede spazio su disco.

Operazione di Vacuum: il comando `deltaTable.vacuum(168.0)` elimina fisicamente tutti i file di dati che non sono più necessari per la versione corrente e che sono più vecchi di **7 giorni (168 ore)**. Questa operazione è fondamentale per:

1. **Risparmio sui costi di storage:** evita che il database cresca indefinitamente.
2. **Conformità normativa:** garantisce che i dati eliminati o modificati non siano recuperabili oltre la finestra di retention stabilita.

8. Risultati e Conclusioni

L'output del `RealTimeAlertManager` ha intercettato pattern di consumo estremamente diversificati. L'utilizzo dello Z-Score dinamico ha permesso di isolare eventi con una confidenza statistica quasi assoluta.

I log evidenziano casi di particolare rilevanza tecnica:

- **MAC001785:** nel Batch 30 è stato rilevato un picco con uno **Z-Score di 29,9**. In una distribuzione normale, un valore oltre le 3 deviazioni standard è raro; un valore vicino a 30 è un evento statisticamente impossibile per fluttuazioni naturali. Con una magnitudo di **52,06 volte superiore alla media attesa** (Severity), questo record identifica con certezza un guasto elettrico o un'attivazione massiva di carichi industriali in ambiente domestico.
- **MAC002670:** nel Batch 45 si osserva una sequenza ininterrotta di alert per oltre 9 ore (dalle 12:00 alle 21:30). La stabilità dello Z-Score (oscillante tra 10,2 e 13,6) indica che l'utente ha mantenuto un carico critico costante, superando la soglia fisica di 6.0 kWh/hh. Questo comportamento è tipico di sistemi di riscaldamento difettosi o ricariche di veicoli elettrici ad alta potenza non regolate.

Il campo `processed_at` mostra che la notifica avviene entro pochi millisecondi dalla fine dell'elaborazione del batch di Spark. Questo dimostra che l'architettura è idonea per sistemi di **Early Warning** in ambito Utility.

L'analisi del Silver Layer ha permesso di mappare l'elasticità energetica dei tre gruppi ACORN (Affluent, Comfortable, Adversity).

I dati mostrano una **correlazione negativa** tra temperatura e consumo per tutti i gruppi, confermando che il riscaldamento elettrico è un driver primario.

Gruppo ACORN	Pearson Correlation (r)	Thermal Slope (kWh/°C)
Affluent	-0.912	0.0573
Comfortable	-0.700	0.0425
Adversity	-0.407	0.0366

Il gruppo **Affluent** ha la sensibilità termica più elevata (-0.9). La pendenza termica di 0.057 indica che per ogni grado di abbassamento della temperatura, il consumo medio aumenta di quasi il doppio rispetto al gruppo Adversity (0.036). Questo suggerisce che le fasce più abbienti riscaldano volumi abitativi maggiori o hanno una minore propensione al risparmio termico.

Imprevedibilità (CoV): il gruppo **Affluent** presenta il `volatility_index` più alto (**1.46**). Questo significa che il loro profilo di carico è meno "piatto" e più soggetto a picchi improvvisi, rappresentando la sfida maggiore per il bilanciamento della rete.

Abitudini festive: il gruppo **Adversity** mostra il maggior incremento di consumo nel weekend (+**6.05%**), seguito dai Comfortable (+**5.95%**). Il gruppo Affluent è il più stabile (+**2.84%**), indicando uno stile di vita dove il consumo energetico è meno vincolato ai giorni lavorativi.

Il Gold Layer ha evidenziato criticità infrastrutturali attraverso l'indice PAR (Peak-to-Average Ratio). I risultati del PAR Index rivelano un dato inaspettato:

- **Ore 03:00 (PAR = 12.31):** è il valore più alto registrato. Nonostante il carico totale della città sia al minimo, poche utenze attive con carichi massicci creano uno squilibrio estremo.
- **Ore 19:00 (uso della rete):** è il momento di massimo carico reale per contatore (0.71 kWh/meter per il gruppo Affluent).

Questo divario indica che la rete è "stressata" in due modi diversi: dal volume totale la sera e dallo squilibrio statistico la notte.

L'unione dei dati analitici con lo storico degli alert ha prodotto le evidenze più significative della ricerca.

L'analisi dei "bin" di temperatura conferma la correttezza della logica context-aware:

- **Fascia 10°C:** 140 alert.
- **Fascia 5°C:** 80 alert.

Il sistema ha dimostrato di essere estremamente vigile nelle mezze stagioni, dove un picco di consumo è meno giustificabile rispetto alle ondate di gelo o caldo intenso (dove la soglia $Z > 15$ ha correttamente filtrato i falsi positivi).

Il risultato più eclatante è la distribuzione delle anomalie per gruppo sociale:

- **Affluent:** ~5.930.000 segnalazioni.
- **Comfortable:** ~570.000 segnalazioni.
- **Adversity:** ~30.000 segnalazioni.

Questa sproporzione indica che il gruppo **Affluent** non solo consuma di più, ma lo fa in modo molto più irregolare e "aggressivo" per la rete. Questo dato suggerisce che le politiche di *Demand Response* dovrebbero essere focalizzate prioritariamente sui quartieri ad alto reddito per ottenere il massimo impatto sulla stabilizzazione della rete.

Sebbene l'architettura proposta sia solida e performante, rappresenta una base per sviluppi futuri come:

1. Integrazione di Machine Learning Predittivo

Il sistema attuale eccelle nella diagnostica *ex-post* (rilevamento del picco quando avviene). Il prossimo passo naturale è l'integrazione di modelli di **Forecasting** (come reti neurali LSTM o modelli Prophet) per prevedere il carico della rete con 24 ore di anticipo. Questo permetterebbe una gestione proattiva delle riserve energetiche invece che meramente reattiva.

2. Automazione del Demand Response

Il modulo degli Alert potrebbe essere collegato direttamente a sistemi di controllo IoT. In presenza di un alert critico e di uno stress di rete elevato (PAR alto), il sistema potrebbe inviare comandi di **Load Shedding** automatico (es. distacco temporaneo di carichi non prioritari come ricariche EV o pompe di calore), previo consenso dell'utente, per prevenire blackout locali.

Il presente lavoro dimostra la fattibilità e l'efficacia di un'architettura Big Data moderna per il monitoraggio e la diagnostica avanzata di una **Smart Grid**. L'integrazione di tecnologie come **Apache Spark Structured Streaming** e **Delta Lake** ha permesso di superare i limiti dei sistemi di

monitoraggio tradizionali, trasformando un flusso grezzo di milioni di record in un sistema di supporto alle decisioni (*Decision Support System*) reattivo e analiticamente profondo.

Obiettivi raggiunti

1. **Efficienza computazionale:** l'implementazione dell'**algoritmo di Welford** è stata la chiave per garantire la scalabilità. Calcolando media e varianza in modalità *one-pass*, il sistema ha gestito lo stato di migliaia di contatori con un'occupazione di memoria costante ($O(1)$), garantendo al contempo una stabilità numerica superiore ai metodi statistici classici.
2. **Rilevamento intelligente delle anomalie:** l'introduzione di **soglie Z-Score context-aware** ha risolto il problema dei falsi positivi legati alla stagionalità. Elevando la tolleranza durante i picchi termici, il sistema ha dimostrato di saper distinguere tra un carico "fisiologico" (riscaldamento/climatizzazione) e un'anomalia critica, raggiungendo una precisione chirurgica nella rilevazione di eventi estremi (fino a 52 volte superiori alla baseline).
3. **Insight socio-energetici:** l'analisi del **Gold Layer** ha rivelato una correlazione netta tra classe socio-economica e stress della rete. Il dato secondo cui il gruppo *Affluent* genera la maggior parte delle anomalie e presenta la volatilità più alta fornisce ai gestori di rete indicazioni precise su dove concentrare gli investimenti di manutenzione e le campagne di efficientamento energetico.